

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Программирование на языке Java»
на тему «Работа с файлами»
Вариант 8

Выполнил:
студент группы 23ВВП1
Батайкин Д.В.
Володин А.П.

Приняли:
Карамышева Н.С.
Юрова О.В.

Цель работы: изучить работу с файлами и механизмы сериализации данных.

Задание на лабораторную работу: Модифицировать приложение из предыдущей лабораторной работы, реализовав сохранение в файл и загрузку данных из файла. Предусмотреть сохранение данных, как в текстовом виде, так и в двоичном (с использованием механизма сериализации). Для этого нужно добавить 4 кнопки для сохранения и загрузки в текстовом и двоичном виде соответственно. Кроме того, в программе нужно предусмотреть использование стандартного диалога открытия файла (JFileChooser). Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

1. Обновили оболочку формы, добавив 4 кнопки для работы с файлами библиотеки Swing (Рис. 1).

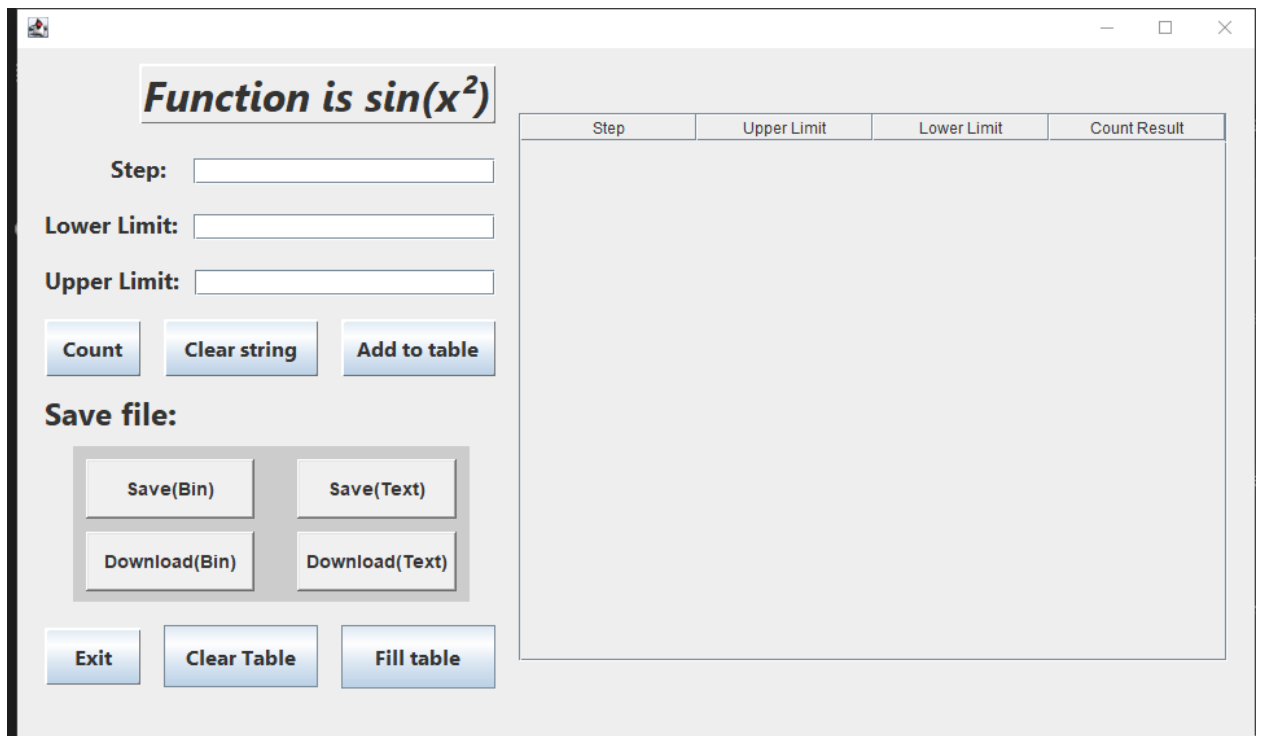


Рисунок 1 – Кнопки для работы с файлами

2. Открытие окна jFileChooser (Рис. 2).

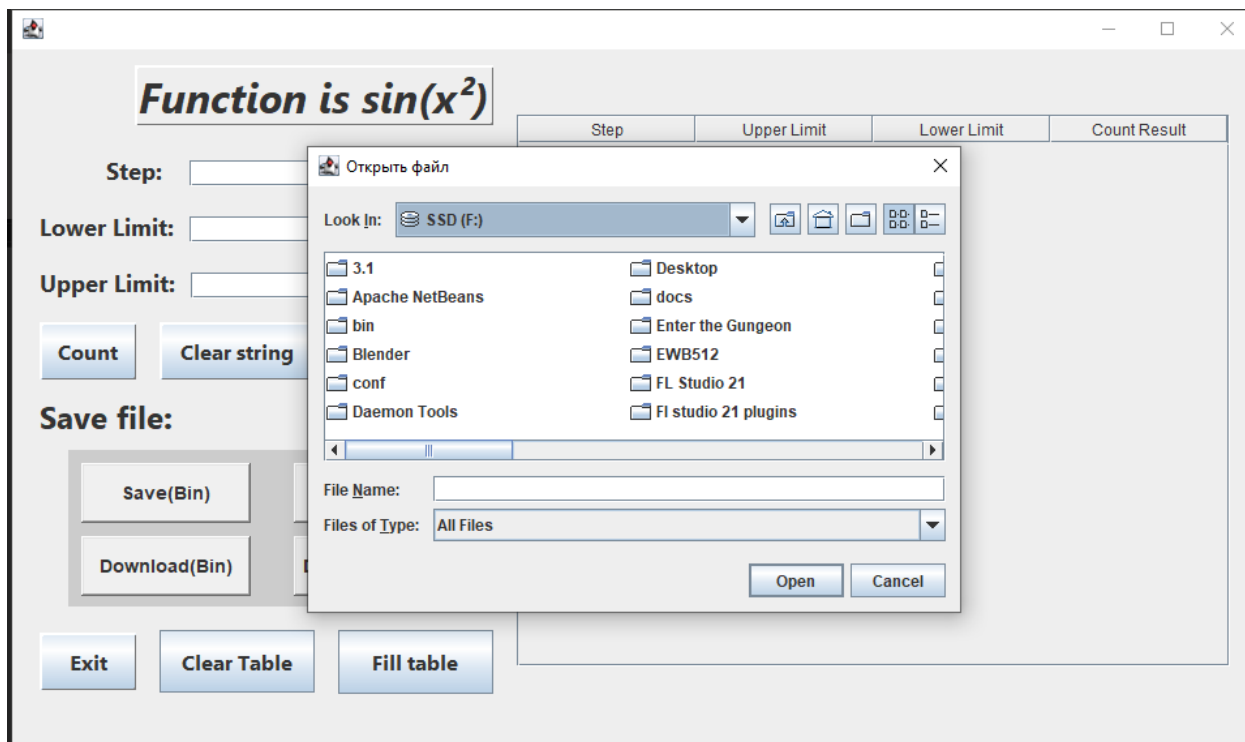


Рисунок 2 – окно jFileChooser

Результат работы:

1. Файлы успешно сохраняются и загружаются в программу (Рис. 3-4).

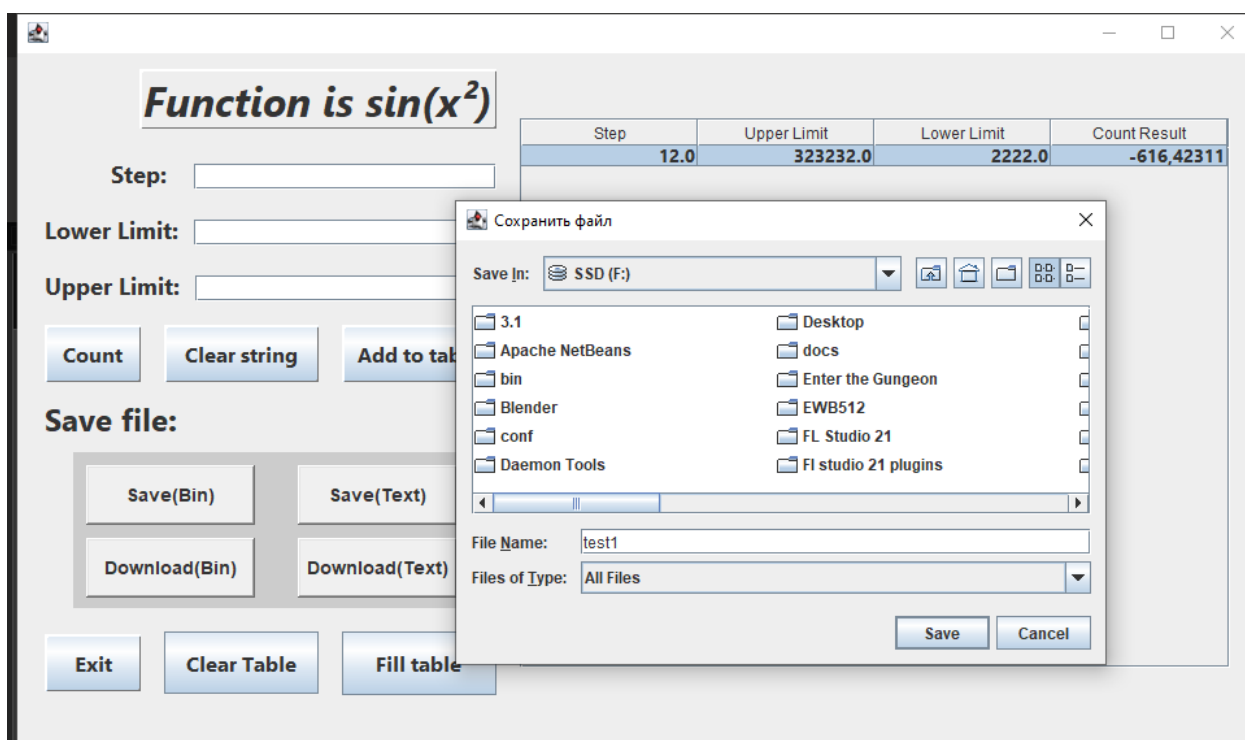


Рисунок 3 – Выбор пути сохранения файла

2. Загрузка файла в таблицу (Рис. 2).

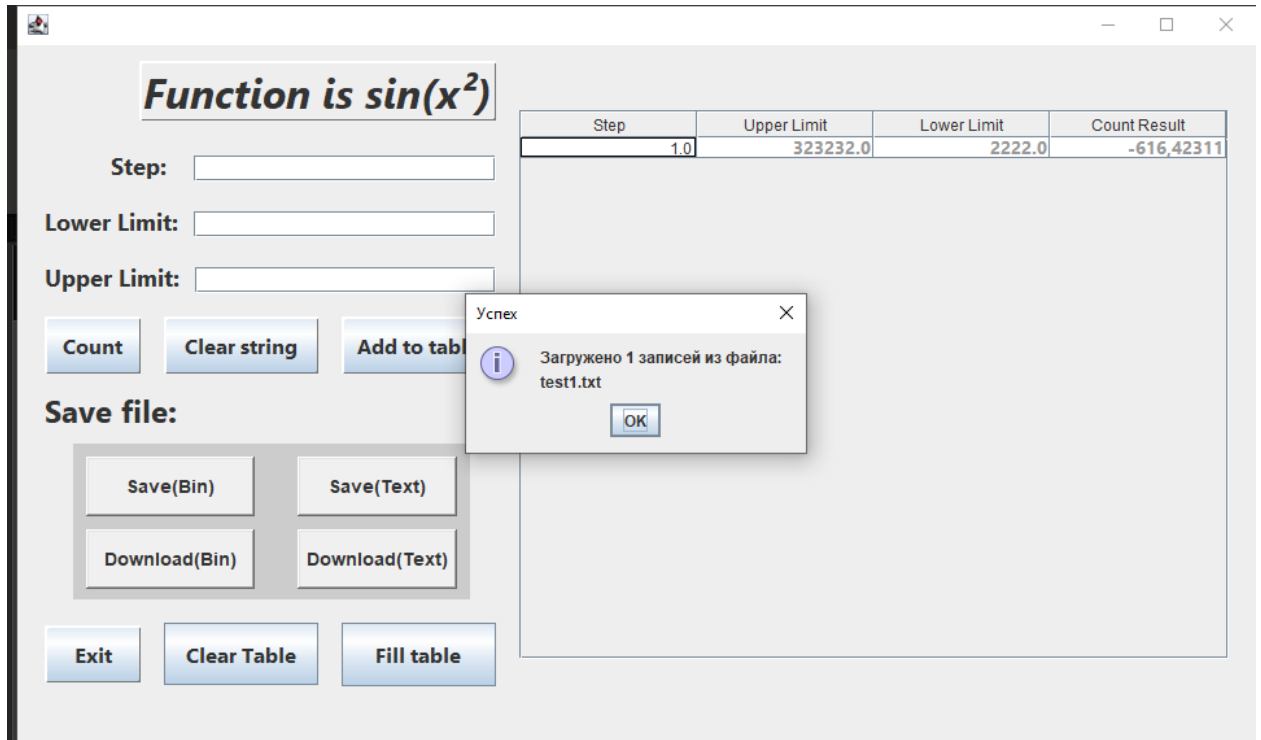


Рисунок 4 – Загрузка файла в таблицу

Вывод: При выполнении данной лабораторной работы был изучен принцип работы с файлами и механизмы сериализации данных, а также усовершенствован интерфейс программы из прошлых работ.

Листинг программы:

Файл [Lab4.java](#)

```
package com.mycompany.lab4;

public class Lab4
{
    public static void main(String[] args) {
        java.awt.EventQueue.invokeLater(() -> {
            new Lab3JFrame().setVisible(true);
        });
    }
}
```

Файл [Lab3JFrame.java](#)

```
package com.mycompany.lab4;
```

```

import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import java.util.LinkedList;
import java.io.File;
import java.io.IOException;
import javax.swing.JFileChooser;
import javax.swing.filechooser.FileNameExtensionFilter;

public class Lab3JFrame extends javax.swing.JFrame {

    private static final java.util.logging.Logger logger =
java.util.logging.Logger.getLogger(Lab3JFrame.class.getName());
    private LinkedList<RecIntegral> recordsList;

    public Lab3JFrame() {
        initComponents();
        setupTableFormatting();
        setupTableListener();
        recordsList = new LinkedList<>();
    }

    private void setupTableFormatting() {
        // Устанавливаем рендерер для отображения чисел
        javax.swing.table.DefaultTableCellRenderer rightRenderer = new
javax.swing.table.DefaultTableCellRenderer();
        rightRenderer.setHorizontalAlignment(javax.swing.JLabel.RIGHT);

        // Применяем рендерер ко всем столбцам
        for (int i = 0; i < TableModel.getColumnCount(); i++) {

TableModel.getColumnModel().getColumn(i).setCellRenderer(rightRenderer);
        }

    }

    private boolean isUpdating = false; // флаг для предотвращения рекурсивного
вызова ошибки

    private void setupTableListener() {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();

        model.addTableModelListener(e -> {
            if (isUpdating) return;

```

```

        if (e.getType() == javax.swing.event.TableModelEvent.UPDATE) {
            int row = e.getFirstRow();
            int column = e.getColumn();

            if (row >= 0 && column >= 0 && column <= 2 && row <
recordsList.size()) {
                isUpdating = true;
                try {
                    RecIntegral record = recordsList.get(row);
                    // Сохраняем старые значения для восстановления
                    double oldStep = record.getStep();
                    double oldUpper = record.getUpperLimit();
                    double oldLower = record.getLowerLimit();

                    Object newValue = model.getValueAt(row, column);
                    if (newValue != null) {
                        String valueStr =
newValue.toString().trim().replace(',', '.');
                        if (!valueStr.isEmpty()) {
                            double value = Double.parseDouble(valueStr);

                            try {
                                // Валидация и обновление с использованием
класса исключения

                                switch (column) {
                                    case 0:
                                        record.setStep(value);
                                        break;
                                    case 1:
                                        record.setUpperLimit(value);
                                        break;
                                    case 2:
                                        record.setLowerLimit(value);
                                        break;
                                }

                                // проверка соотношения пределов
                                if
(record.getLowerLimit() >=
record.getUpperLimit()) {
                                    JOptionPane.showMessageDialog(this,
"Нижний предел должен быть меньше верхнего", "Ошибка",
JOptionPane.WARNING_MESSAGE);

```

```

        // Восстанавливаем значения
        record.setStep(oldStep);
        record.setUpperLimit(oldUpper);
        record.setLowerLimit(oldLower);

        // Обновляем таблицу
        model.setValueAt(oldStep, row, 0);
        model.setValueAt(oldUpper, row, 1);
        model.setValueAt(oldLower, row, 2);
    } else {
        record.setResult(0.0);
        model.setValueAt("", row, 3);
    }

} catch (IntegralException ex) {
    // Обработка исключения при редактировании
ячейки

    String fieldName = "";
    double restoreValue = 0;

    switch (column) {
        case 0:
            fieldName = "Шаг";
            restoreValue = oldStep;
            break;
        case 1:
            fieldName = "Верхний предел";
            restoreValue = oldUpper;
            break;
        case 2:
            fieldName = "Нижний предел";
            restoreValue = oldLower;
            break;
    }

    JOptionPane.showMessageDialog(this, String.format("Некорректное значение для
поля '%s': %.6f\n\n" + "Значение должно быть в диапазоне от %.6f до %.6f",
        fieldName, value,
        RecIntegral.MIN_VALUE, RecIntegral.MAX_VALUE), "Ошибка
валидации", JOptionPane.WARNING_MESSAGE);

    // Восстанавливаем старое значение
    model.setValueAt(restoreValue, row, column);

```

```

        }
    }
}

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(this, "Некорректное число",
    "Ошибка формата", JOptionPane.ERROR_MESSAGE);

    // Восстанавливаем старое значение
    RecIntegral record = recordsList.get(row);
    double restoreValue = column == 0 ? record.getStep() :
(column == 1 ? record.getUpperLimit() : record.getLowerLimit());
    model.setValueAt(restoreValue, row, column);
} finally {
    isUpdating = false;
}
}
}

});
}

//методы для файлов
private void showError(String message, Exception e) {
    logger.severe(message + ": " + e.getMessage());
    JOptionPane.showMessageDialog(this, message + "\n\n" + e.getMessage(),
    "Ошибка", JOptionPane.ERROR_MESSAGE);
}

private void updateTableFromList() {
    DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
    model.setRowCount(0); // Очищаем таблицу
    for (RecIntegral record : recordsList) {
        model.addRow(record.toTableRow());
    }
}

private File chooseFile(boolean forSave, String description, String...
extensions) {
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setDialogTitle(forSave ? "Сохранить файл" : "Открыть файл");

    // Добавляем фильтры файлов
    for (String ext : extensions) {

```



```

        fileChooser.addChoosableFileFilter(
            new
            javax.swing.filechooser.FileNameExtensionFilter(ext.toUpperCase() + " файлы
            (*.\" + ext + \")\", ext));
    }

    int result = forSave ? fileChooser.showSaveDialog(this) :
    fileChooser.showOpenDialog(this);

    if (result == JFileChooser.APPROVE_OPTION) {
        File selectedFile = fileChooser.getSelectedFile();

        // Добавляем расширение, если его нет
        if (forSave && selectedFile.getName().indexOf('.') == -1) {
            String ext = extensions[0];
            selectedFile = new File(selectedFile.getAbsolutePath() + "." +
ext);
        }

        return selectedFile;
    }

    return null;
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">
private void initComponents() {

    jPanel1 = new javax.swing.JPanel();
    jPanel2 = new javax.swing.JPanel();
    jLabel1 = new javax.swing.JLabel();
    jScrollPane1 = new javax.swing.JScrollPane();
    TableModel = new javax.swing.JTable();
    CountButton = new javax.swing.JButton();
    ClearStringButton = new javax.swing.JButton();
    AddToTableButton = new javax.swing.JButton();
    StepLabel = new javax.swing.JLabel();
    UpLimitLabel = new javax.swing.JLabel();
    DownLimitLabel = new javax.swing.JLabel();
    StepTextField = new javax.swing.JTextField();
    UpperLimitTextField = new javax.swing.JTextField();
    LowerLimitTextField = new javax.swing.JTextField();
    ExitButton = new javax.swing.JButton();

```

```

ClearTableButton = new javax.swing.JButton();
FillTableButton = new javax.swing.JButton();
FilePane = new java.awt.Panel();
SaveFileBin = new java.awt.Button();
SaveFileText = new java.awt.Button();
DownloadFileBin = new java.awt.Button();
DownloadFileText = new java.awt.Button();
FilePaneLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jLabel1.setFont(new java.awt.Font("Segoe UI", 3, 32)); // NOI18N
jLabel1.setText("Function is  $\sin(x^2)$ ");
jLabel1.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));

TableModel.setBorder(javax.swing.BorderFactory.createCompoundBorder());
TableModel.setFont(new java.awt.Font("Segoe UI", 1, 14)); // NOI18N
TableModel.setForeground(new java.awt.Color(153, 153, 153));
TableModel.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
    new String [] {
        "Step", "Upper Limit", "Lower Limit", "Count Result"
    }
) {
    Class[] types = new Class [] {
        java.lang.Double.class,                java.lang.Double.class,
        java.lang.Double.class, java.lang.Double.class
    };
    boolean[] canEdit = new boolean [] {
        true, true, true, false
    };

    public Class getColumnClass(int columnIndex) {
        return types [columnIndex];
    }

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
}

```

```

    });
    TableModel.setToolTipText("");
    TableModel.setShowVerticalLines(true);
    jScrollPane1.setViewportViewView(TableModel);

    CountButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
    CountButton.setText("Count");
    CountButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
    CountButton.addActionListener(this::CountButtonActionPerformed);

    ClearStringButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); //
NOI18N
    ClearStringButton.setText("Clear string");
    ClearStringButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));

    ClearStringButton.addActionListener(this::ClearStringButtonActionPerformed);

    AddToTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); //
NOI18N
    AddToTableButton.setText("Add to table");
    AddToTableButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));

    AddToTableButton.addActionListener(this::AddToTableButtonActionPerformed);

    StepLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
    StepLabel.setText("Step:");

    UpLimitLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
    UpLimitLabel.setText("Upper Limit:");

    DownLimitLabel.setFont(new java.awt.Font("Segoe UI", 1, 18)); // NOI18N
    DownLimitLabel.setText("Lower Limit:");

    StepTextField.addActionListener(this::StepTextFieldActionPerformed);

    UpperLimitTextField.addActionListener(this::UpperLimitTextFieldActionPerforme
d);

```

```
LowerLimitTextField.addActionListener(this::LowerLimitTextFieldActionPerformed);
```

```
ExitButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); // NOI18N
ExitButton.setText("Exit");
ExitButton.setBorder(new
javax.swing.border.SoftBevelBorder(javax.swing.border.BevelBorder.RAISED));
ExitButton.addActionListener(this::ExitButtonActionPerformed);
```

```
ClearTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); //
NOI18N
ClearTableButton.setText("Clear Table");
```

```
ClearTableButton.addActionListener(this::ClearTableButtonActionPerformed);
```

```
FillTableButton.setFont(new java.awt.Font("Segoe UI", 1, 16)); //
NOI18N
FillTableButton.setText("Fill table");
```

```
FillTableButton.addActionListener(this::FillTableButtonActionPerformed);
```

```
FilePane.setBackground(new java.awt.Color(204, 204, 204));
FilePane.setFont(new java.awt.Font("Dialog", 1, 16)); // NOI18N
```

```
SaveFileBin.setActionCommand("SaveFileBin");
SaveFileBin.setFont(new java.awt.Font("Dialog", 1, 14)); // NOI18N
SaveFileBin.setLabel("Save (Bin)");
SaveFileBin.addActionListener(this::SaveFileBinActionPerformed);
```

```
SaveFileText.setActionCommand("SaveFileText");
SaveFileText.setFont(new java.awt.Font("Dialog", 1, 14)); // NOI18N
SaveFileText.setLabel("Save (Text)");
SaveFileText.addActionListener(this::SaveFileTextActionPerformed);
```

```
DownloadFileBin.setActionCommand("DownloadFileBin");
DownloadFileBin.setFont(new java.awt.Font("Dialog", 1, 14)); // NOI18N
DownloadFileBin.setLabel("Download (Bin)");
```

```
DownloadFileBin.addActionListener(this::DownloadFileBinActionPerformed);
```

```
DownloadFileText.setActionCommand("DownloadFileText");
DownloadFileText.setFont(new java.awt.Font("Dialog", 1, 14)); // NOI18N
```

```

DownloadFileText.setLabel("Download(Text)");

DownloadFileText.addActionListener(this::DownloadFileTextActionPerformed);

        javax.swing.GroupLayout FilePaneLayout = new
javax.swing.GroupLayout(FilePane);
        FilePane.setLayout(FilePaneLayout);
        FilePaneLayout.setHorizontalGroup(

FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(FilePaneLayout.createSequentialGroup()
                .addGap(
                    .addGroup(FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
                        .addComponent(DownloadFileBin,
javax.swing.GroupLayout.DEFAULT_SIZE, 130, Short.MAX_VALUE)
                        .addComponent(SaveFileBin,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
                    .addGap(33,
Short.MAX_VALUE)
                    .addGroup(FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
                        .addComponent(DownloadFileText,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)
                        .addComponent(SaveFileText,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
                    .addGap(
                );
        FilePaneLayout.setVerticalGroup(

FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(FilePaneLayout.createSequentialGroup()
                .addGap(
                    .addGroup(FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)

```

```

        .addComponent(SaveFileBin,
javax.swing.GroupLayout.DEFAULT_SIZE, 46, Short.MAX_VALUE)
        .addComponent(SaveFileText,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

.addGroup(FilePaneLayout.createParallelGroup(javax.swing.GroupLayout.Alignmen
t.LEADING, false)
        .addComponent(DownloadFileBin,
javax.swing.GroupLayout.DEFAULT_SIZE, 46, Short.MAX_VALUE)
        .addComponent(DownloadFileText,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))
        .addGap(39, 39, 39))
    );

    SaveFileBin.getAccessibleContext().setAccessibleName("SaveFileBin");

    SaveFileText.getAccessibleContext().setAccessibleName("SaveFileText");

    DownloadFileBin.getAccessibleContext().setAccessibleName("DownloadFileBin");

    DownloadFileText.getAccessibleContext().setAccessibleName("DownloadFileText")
;

    FilePaneLabel.setFont(new java.awt.Font("Segoe UI", 1, 24)); // NOI18N
    FilePaneLabel.setText("Save file:");

    javax.swing.GroupLayout jPanel2Layout = new
javax.swing.GroupLayout(jPanel2);
    jPanel2.setLayout(jPanel2Layout);
    jPanel2Layout.setHorizontalGroup(

jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel2Layout.createSequentialGroup())

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

        .addGroup(jPanel2Layout.createSequentialGroup())
        .addGap(21, 21, 21)

```

```

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

.addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanel2Layout.createSequentialGroup())

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.TRAILING)

                                .addComponent(DownLimitLabel)
                                .addComponent(StepLabel,
javax.swing.GroupLayout.PREFERRED_SIZE,                    52,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

                                .addComponent(LowerLimitTextField)
                                .addComponent(StepTextField))

.addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanel2Layout.createSequentialGroup()

                                .addGap(0, 73, Short.MAX_VALUE)
                                .addComponent(jLabel1))
.addGroup(jPanel2Layout.createSequentialGroup())

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING)

                                .addComponent(CountButton,
javax.swing.GroupLayout.DEFAULT_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

                                .addComponent(ExitButton,
javax.swing.GroupLayout.DEFAULT_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))

                                .addGap(18, 18, 18)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.LEADING, false)

                                .addComponent(ClearStringButton,
javax.swing.GroupLayout.DEFAULT_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)

```

```

        .addComponent(ClearTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE))

        .addGap(18, 18, 18)

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
    .LEADING, false)

        .addComponent(AddToTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE, 119, Short.MAX_VALUE)
        .addComponent(FillTableButton,
javax.swing.GroupLayout.DEFAULT_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
Short.MAX_VALUE)))

        .addGroup(jPanel2Layout.createSequentialGroup())
        .addComponent(UpLimitLabel)

    .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(UpperLimitTextField)
        .addGroup(jPanel2Layout.createSequentialGroup())
        .addComponent(FilePaneLabel)
        .addGap(0, 0, Short.MAX_VALUE)))
        .addGap(18, 18, 18))
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,
jPanel2Layout.createSequentialGroup())

    .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(FilePane,
javax.swing.GroupLayout.PREFERRED_SIZE,    javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(38, 38, 38))
        .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE,                                     547,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(21, 21, 21))
    );
    jPanel2Layout.setVerticalGroup(

jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(jPanel2Layout.createSequentialGroup())

    .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
    .LEADING)

        .addGroup(jPanel2Layout.createSequentialGroup())
        .addGap(50, 50, 50)

```



```

        .addComponent(jScrollPane1,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGroup(jPanel2Layout.createSequentialGroup())
        .addContainerGap()
        .addComponent(jLabel1,
javax.swing.GroupLayout.PREFERRED_SIZE, 46,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(23, 23, 23)

        .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASILINE)
        .addComponent(StepTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(StepLabel,
javax.swing.GroupLayout.PREFERRED_SIZE, 22,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

        .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASILINE)
        .addComponent(DownLimitLabel)
        .addComponent(LowerLimitTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

        .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASILINE)
        .addComponent(UpLimitLabel)
        .addComponent(UpperLimitTextField,
javax.swing.GroupLayout.PREFERRED_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addGap(18, 18, 18)

        .addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASILINE)
        .addComponent(CountButton,
javax.swing.GroupLayout.PREFERRED_SIZE, 44,
javax.swing.GroupLayout.PREFERRED_SIZE)

```

```

        .addComponent(ClearStringButton,
javax.swing.GroupLayout.PREFERRED_SIZE,          44,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(AddToTableButton,
javax.swing.GroupLayout.PREFERRED_SIZE,          44,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
        .addComponent(FilePaneLabel)

.addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(FilePane,
javax.swing.GroupLayout.PREFERRED_SIZE,          120,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(18, 18, 18)

.addGroup(jPanel2Layout.createParallelGroup(javax.swing.GroupLayout.Alignment
.BASELINE)
        .addComponent(ExitButton,
javax.swing.GroupLayout.PREFERRED_SIZE,          44,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(FillTableButton,
javax.swing.GroupLayout.PREFERRED_SIZE,          49,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(ClearTableButton,
javax.swing.GroupLayout.PREFERRED_SIZE,          48,
javax.swing.GroupLayout.PREFERRED_SIZE)))
        .addContainerGap(42, Short.MAX_VALUE))
    );

```

```
FilePaneLabel.getAccessibleContext().setAccessibleName("FilePaneLabel");
```

```

        javax.swing.GroupLayout jPanel1Layout = new
javax.swing.GroupLayout(jPanel1);
        jPanel1.setLayout(jPanel1Layout);
        jPanel1Layout.setHorizontalGroup(

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addComponent(jPanel2, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        );
        jPanel1Layout.setVerticalGroup(

```

```

jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jPanel2, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    javax.swing.GroupLayout layout = new
javax.swing.GroupLayout(getContentPane());
    getContentPane().setLayout(layout);
    layout.setHorizontalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(

layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold>

private void CountButtonActionPerformed(java.awt.event.ActionEvent evt) {
    int selectedRow = TableModel.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(this, "Выберите строку для
вычисления", "Информация", JOptionPane.WARNING_MESSAGE);
        return;
    }
    try{
        DefaultTableModel model = (DefaultTableModel)
TableModel.getModel();
        // Получаем значения из таблицы
        Double step = (Double) TableModel.getValueAt(selectedRow, 0);
        Double uplim = (Double) TableModel.getValueAt(selectedRow, 1);
        Double lowlim = (Double) TableModel.getValueAt(selectedRow, 2);

        // Проверяем на null

```

```

        if (step == null || uplim == null || lowlim == null) {
JOptionPane.showMessageDialog(this, "Некоторые значения отсутствуют",
"Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }
        // Проверка корректности параметров
        if (lowlim >= uplim) {
            JOptionPane.showMessageDialog(this, "Нижний предел должен быть
меньше верхнего предела", "Ошибка параметров", JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (step <= 0) {JOptionPane.showMessageDialog(this, "Шаг должен
быть положительным числом", "Ошибка параметров", JOptionPane.ERROR_MESSAGE);
            return;
        }

        double result = RecIntegral.computeIntegral(step, lowlim, uplim);

        // обновляем результат в коллекции
        if (selectedRow < recordsList.size()) {
            RecIntegral record = recordsList.get(selectedRow);
            record.setResult(result);
        }

        // Форматируем результат до 5 знаков
        model.setValueAt(String.format("%.5f", result), selectedRow, 3);
    }
    catch(ClassCastException e) {
        // обработка неверного формата
        JOptionPane.showMessageDialog(this, "Некорректный формат данных в
таблице", "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
    catch(Exception e){
        JOptionPane.showMessageDialog(this, "Ошибка при вычислении: " +
e.getMessage(), "Ошибка", JOptionPane.ERROR_MESSAGE);
    }
}

private void ClearStringButtonActionPerformed(java.awt.event.ActionEvent
evt) {
    int selectedRow = TableModel.getSelectedRow();
    if (selectedRow == -1) {

```

```

        JOptionPane.showMessageDialog(this, "Выберите строку для
удаления", "Информация", JOptionPane.WARNING_MESSAGE);
        return;
    }
    // получаем модель и удаляем строку из таблицы и коллекции
    DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
    model.removeRow(selectedRow);
    recordsList.remove(selectedRow);
}

private void AddToTableButtonActionPerformed(java.awt.event.ActionEvent
evt) {
    try {
        String stepText = StepTextField.getText().trim().replace(',', '
'.'); // Поддержка запятой
        String lowlimText =
LowerLimitTextField.getText().trim().replace(',', ' '.');
        String uplimText =
UpperLimitTextField.getText().trim().replace(',', ' '.');

        // проверка на заполнение полей ввода
        if (stepText.isEmpty() || lowlimText.isEmpty() || uplimText.isEmpty())
        {
            JOptionPane.showMessageDialog(this, "Заполните все поля",
"Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        double step, lowlim, uplim;
        try{
            step = Double.parseDouble(stepText);
            lowlim = Double.parseDouble(lowlimText);
            uplim = Double.parseDouble(uplimText);
        }
        catch (NumberFormatException e)
        {JOptionPane.showMessageDialog(this, "Ошибка парсинга переменных.",
"Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Проверка диапазона значений (0.000001 до 1000000)

```

```

        if (step < RecIntegral.MIN_VALUE || step > RecIntegral.MAX_VALUE ||
lowlim < RecIntegral.MIN_VALUE || lowlim > RecIntegral.MAX_VALUE || uplim <
RecIntegral.MIN_VALUE || uplim > RecIntegral.MAX_VALUE) {
            String message = String.format( "Значения должны быть в
диапазоне от %.6f до %.6f\n\n" + "Введенные значения:\n" + "Шаг: %.6f\n" +
"Нижний предел: %.6f\n" + "Верхний предел: %.6f",
                RecIntegral.MIN_VALUE, RecIntegral.MAX_VALUE, step, lowlim,
uplim);

            JOptionPane.showMessageDialog(this, message, "Ошибка
валидации", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (lowlim >= uplim) {
            JOptionPane.showMessageDialog(this, "Нижний предел должен быть
меньше верхнего предела", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (step <= 0) {JOptionPane.showMessageDialog(this, "Шаг должен
быть положительным числом", "Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }
        if (step > uplim - lowlim) {JOptionPane.showMessageDialog(this,
"Шаг должен быть меньше или равен разнице верхнего и нижнего пределов",
"Ошибка", JOptionPane.ERROR_MESSAGE);
            return;
        }

        // Создаем объект (конструктор без исключений)
        RecIntegral record = new RecIntegral(step, uplim, lowlim);

        // добавляем в список
        recordsList.add(record);
        // добавление в таблицу
        DefaultTableModel model = (DefaultTableModel)
TableModel.getModel();
        model.addRow(record.toTableRow());

        // Очищаем поля
        StepTextField.setText("");
        LowerLimitTextField.setText("");
        UpperLimitTextField.setText("");

```

```

        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Введите корректные числовые значения (например: 0.1, 0.5, 1.0)", "Ошибка ввода", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void StepTextFieldActionPerformed(java.awt.event.ActionEvent evt)
    {

    }

    private void UpperLimitTextFieldActionPerformed(java.awt.event.ActionEvent evt) {

    }

    private void LowerLimitTextFieldActionPerformed(java.awt.event.ActionEvent evt) {

    }

    private void ExitButtonActionPerformed(java.awt.event.ActionEvent evt) {
        System.exit(0);
    }

    private void ClearTableButtonActionPerformed(java.awt.event.ActionEvent evt) {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();
        model.setRowCount(0);
    }

    private void FillTableButtonActionPerformed(java.awt.event.ActionEvent evt) {
        DefaultTableModel model = (DefaultTableModel) TableModel.getModel();

        if (recordsList.isEmpty()) {JOptionPane.showMessageDialog(this, "Коллекция пуста. Сначала добавьте данные", "Информация", JOptionPane.INFORMATION_MESSAGE);
            return;}

        int rowCount = model.getRowCount();
        for (int i = rowCount - 1; i >= 0; i--) {

```

```

        model.removeRow(i);
    }
    for (RecIntegral record : recordsList) {
        model.addRow(record.toTableRow());
    }
}

private void SaveFileBinActionPerformed(java.awt.event.ActionEvent evt) {
    if (recordsList.isEmpty()) {JOptionPane.showMessageDialog(this, "Нет
данных для сохранения", "Информация", JOptionPane.INFORMATION_MESSAGE);
    return;
}

File file = chooseFile(true, "Выберите файл для сохранения", "bin", "ser");
if (file == null) return;

try {
    FileManager.saveToBinaryFile(recordsList, file);
    JOptionPane.showMessageDialog(this, "Данные успешно сохранены в
бинарный файл:\n" + file.getName(), "Успех",
    JOptionPane.INFORMATION_MESSAGE);
} catch (IntegralException e) {
    showError("Ошибка валидации данных при сохранении", e);
} catch (IOException e) {
    showError("Ошибка при сохранении бинарного файла", e);
}
}

private void DownloadFileBinActionPerformed(java.awt.event.ActionEvent
evt) {
    File file = chooseFile(false, "Выберите бинарный файл для загрузки",
"bin", "ser");
    if (file == null) return;

    try {
        LinkedList<RecIntegral> loadedRecords =
FileManager.loadFromBinaryFile(file);

        // Спрашиваем пользователя, что делать с загруженными данными
        int choice = JOptionPane.showConfirmDialog(this, "Загруженные данные
будут добавлены к
текущим.\nПродолжить?", "Подтверждение", JOptionPane.YES_NO_OPTION);

```



```

        if (choice == JOptionPane.YES_OPTION) {
            recordsList.addAll(loadedRecords);
            updateTableFromList();
            JOptionPane.showMessageDialog(this, String.format("Загружено %d
записей из файла:\n%s", loadedRecords.size(), file.getName()), "Успех",
JOptionPane.INFORMATION_MESSAGE);
        }

    } catch (ClassNotFoundException e) {
        showError("Неверный формат файла или версия класса", e);
    } catch (IntegralException e) {
        showError("Ошибка валидации загруженных данных", e);
    } catch (IOException e) {
        showError("Ошибка при загрузке бинарного файла", e);
    }
}

private void SaveFileTextActionPerformed(java.awt.event.ActionEvent evt) {
    if (recordsList.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Нет данных для сохранения",
"Информация", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    File file = chooseFile(true, "Выберите файл для сохранения", "txt", "csv");
    if (file == null) return;

    try {
        FileManager.saveToTextFile(recordsList, file);
        JOptionPane.showMessageDialog(this, "Данные успешно сохранены в
текстовый файл:\n" + file.getName(), "Успех", JOptionPane.INFORMATION_MESSAGE);
    } catch (IntegralException e) {
        showError("Ошибка валидации данных при сохранении", e);
    } catch (IOException e) {
        showError("Ошибка при сохранении текстового файла", e);
    }
}

private void DownloadFileTextActionPerformed(java.awt.event.ActionEvent
evt) {
    File file = chooseFile(false, "Выберите текстовый файл для загрузки",
"txt", "csv");
    if (file == null) return;

```

```

        try {
            LinkedList<RecIntegral> loadedRecords =
FileManager.loadFromTextFile(file);

            int choice = JOptionPane.showConfirmDialog(this, "Загруженные данные
будут добавлены к текущим.\nПродолжить?", "Подтверждение",
JOptionPane.YES_NO_OPTION);

            if (choice == JOptionPane.YES_OPTION) {
                recordsList.addAll(loadedRecords);
                updateTableFromList();
                JOptionPane.showMessageDialog(this, String.format("Загружено %d
записей из файла:\n%s", loadedRecords.size(), file.getName()), "Успех",
JOptionPane.INFORMATION_MESSAGE);
            }

        } catch (IntegralException e) {
            showError("Ошибка валидации загруженных данных", e);
        } catch (IOException e) {
            showError("Ошибка при загрузке текстового файла", e);
        } catch (Exception e) {
            showError("Неожиданная ошибка при загрузке файла", e);
        }
    }

    public static void main(String args[]) {
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception e) {
            java.awt.EventQueue.invokeLater(() -> new
Lab3JFrame().setVisible(true));
        }
        // Variables declaration - do not modify

```

```

private javax.swing.JButton AddToTableButton;
private javax.swing.JButton ClearStringButton;
private javax.swing.JButton ClearTableButton;
private javax.swing.JButton CountButton;
private javax.swing.JLabel DownLimitLabel;
private java.awt.Button DownloadFileBin;
private java.awt.Button DownloadFileText;
private javax.swing.JButton ExitButton;
private java.awt.Panel FilePane;
private javax.swing.JLabel FilePaneLabel;
private javax.swing.JButton FillTableButton;
private javax.swing.JTextField LowerLimitTextField;
private java.awt.Button SaveFileBin;
private java.awt.Button SaveFileText;
private javax.swing.JLabel StepLabel;
private javax.swing.JTextField StepTextField;
private javax.swing.JTable TableModel;
private javax.swing.JLabel UpLimitLabel;
private javax.swing.JTextField UpperLimitTextField;
private javax.swing.JLabel jLabel1;
private javax.swing.JPanel jPanel1;
private javax.swing.JPanel jPanel2;
private javax.swing.JScrollPane jScrollPane1;
// End of variables declaration
}

```

Файл RecIntegra.java

```
package com.mycompany.lab4;
```

```
import java.io.Serializable;
```

```

public class RecIntegral implements Serializable {
    private static final long serialVersionUID = 1L; // для версионирования

    // константы минимального/максимального значений для вводимых данных
    public static final double MIN_VALUE = 0.000001;
    public static final double MAX_VALUE = 1000000.0;

    // Метод валидации значения
    private static void validateValue(double value, String fieldName) throws
IntegralException {
        if (value < MIN_VALUE || value > MAX_VALUE) {

```

```

        throw new IntegralException(String.format("Значение поля %s (%.6f)
должно быть в диапазоне от %.6f до %.6f", fieldName, value, MIN_VALUE,
MAX_VALUE), fieldName, value);
    }
}

// переменные таблицы
private double step;
private double upperLimit;
private double lowerLimit;
private double result;

// конструктор по умолчанию
public RecIntegral() {this.step = 0.0;
    this.upperLimit = 0.0;
    this.lowerLimit = 0.0;
    this.result = 0.0;
}

// Конструктор с параметрами
public RecIntegral(double step, double upperLimit, double lowerLimit,
double result){
    this.step = step;
    this.upperLimit = upperLimit;
    this.lowerLimit = lowerLimit;
    this.result = result;
}

// Конструктор без результата (для новых записей)
public RecIntegral(double step, double upperLimit, double lowerLimit) {
    this(step, upperLimit, lowerLimit, 0.0);
}

// геттеры и сеттеры переменных RecIntegral + проверка значений (валидация)
public double getStep() { return step;}
public void setStep(double step) throws IntegralException {
    validateValue(step, "Шаг");
    this.step = step;}

public double getUpperLimit() {return upperLimit;}
public void setUpperLimit (double upperLimit) throws IntegralException {
    validateValue(upperLimit, "Верхний предел");
    this.upperLimit = upperLimit;}

public double getLowerLimit() {return lowerLimit;}

```

```

    public void setLowerLimit(double lowerLimit) throws IntegralException {
        validateValue(lowerLimit, "Нижний предел");
        this.lowerLimit = lowerLimit;}

    public double getResult() {return result;}
    public void setResult(double result) {this.result = result;}

    // Метод для преобразования в массив Object для таблицы
    public Object[] toTableRow() {
        return new Object[]{step, upperLimit, lowerLimit, result == 0.0 ? "" :
String.format("%.5f", result)};
    }

    // Вычисление интеграла //
    public static double func(double x) {
        return Math.sin(x*x);
    }

    public static double computeIntegral(double step, double lowlim, double
uplim) {
        double sum = 0.0;
        double x = lowlim;

        while (x < uplim) {
            //Подсчет шага с учетом погрешности с помощью минимизации Math.min
            double currentstep = Math.min(step, uplim - x);
            double nextX = x + currentstep;

            double y1 = func(x);
            double y2 = func(nextX);
            // Площадь трапеции
            sum += (y1 + y2) * (nextX - x) / 2.0;
            x = nextX;
        }
        return sum;
    }
}

package com.mycompany.lab4;

public class IntegralException extends Exception {
    private String fieldName;    // поле, вызвавшее ошибку
    private double invalidValue; // значение, вызвавшее ошибку

```

```

        // конструктор
        public IntegralException(String message, String fieldName, double
invalidValue) {
            // Передаем сообщение в родительский класс Exception
            super(message);
            this.fieldName = fieldName;
            this.invalidValue = invalidValue;
        }

        // конструктор когда необходим только текст об ошибке
        public IntegralException(String message) {super(message);}

        // геттеры
        public String getFieldName() {return fieldName;}

        public double getInvalidValue() {return invalidValue;}

    }

package com.mycompany.lab4;

import java.io.*;
import java.util.LinkedList;

public class FileManager {
    // Сохранение в текстовый файл
    public static void saveToTextFile(LinkedList<RecIntegral> records, File
file) throws IOException, IntegralException {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(file)))
        {
            // заголовок
            writer.write("Step;UpperLimit;LowerLimit;Result");
            writer.newLine();

            for (RecIntegral record : records) {
                // проверка(валидация перед сохранением)
                validateRecord(record);
                String line = String.format("%.6f;%.6f;%.6f;%.6f",
record.getStep(), record.getUpperLimit(), record.getLowerLimit(),
record.getResult()); writer.write(line); writer.newLine();
            }
        }
    }
}

```

```

        // Загрузка из текстового файла
        public static LinkedList<RecIntegral> loadFromTextFile(File file) throws
IOException, IntegralException {
            LinkedList<RecIntegral> records = new LinkedList<>();
            try (BufferedReader reader = new BufferedReader(new FileReader(file)))
            {
                String line;
                int lineNumber = 0;
                // пропуск заголовка
                reader.readLine();

                while ((line = reader.readLine()) != null) {
                    lineNumber++;
                    if (line.trim().isEmpty()) continue; // пропуск пустых строк

                    String[] parts = line.split(";"); // разбив строки на массив
подстрок
                    if (parts.length != 4) {
                        throw new IntegralException(
                            String.format("Неверный формат строки %d: ожидается 4
значения, получено %d", lineNumber, parts.length));
                    }

                    try {
                        double step = Double.parseDouble(parts[0].replace(',', '
.'));

                        double upperLimit =
Double.parseDouble(parts[1].replace(',', '
.'));
                        double lowerLimit =
Double.parseDouble(parts[2].replace(',', '
.'));
                        double result = Double.parseDouble(parts[3].replace(',', '
.'));

                        // Валидация данных
                        validateValues(step, upperLimit, lowerLimit);

                        RecIntegral record = new RecIntegral(step, upperLimit,
lowerLimit, result);
                        records.add(record);

                    } catch (NumberFormatException e) {
                        throw new IntegralException(String.format("Ошибка парсинга
числа в строке %d: %s", lineNumber, e.getMessage()));
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}

return records;
}

// сохранение в бинарный файл (сериализация)
public static void saveToBinaryFile(LinkedList<RecIntegral> records, File
file) throws IOException, IntegralException {
    // Валидация записей перед сохранением
    for (RecIntegral record : records) {
        validateRecord(record);
    }

    try (ObjectOutputStream oos = new ObjectOutputStream(
        new BufferedOutputStream(new FileOutputStream(file))))
    {oos.writeObject(records);
    }
}

// Загрузка из бинарного файла (десериализация)
@SuppressWarnings("unchecked")
public static LinkedList<RecIntegral> loadFromBinaryFile(File file) throws
IOException, ClassNotFoundException, IntegralException {

    try (ObjectInputStream ois = new ObjectInputStream(
        new BufferedInputStream(new FileInputStream(file)))) {

        Object obj = ois.readObject(); // десереализация

        if (!(obj instanceof LinkedList)) {
            throw new IntegralException("Файл содержит неверный тип
данных");
        }

        LinkedList<RecIntegral> records = (LinkedList<RecIntegral>) obj;
        // Валидация загруженных данных
        for (RecIntegral record : records) {
            validateRecord(record);
        }
        return records;
    }
}

```



```

        // методы валидации
        private static void validateRecord(RecIntegral record) throws
IntegralException {validateValues(record.getStep(), record.getUpperLimit(),
record.getLowerLimit());

        // Проверка соотношения пределов
        if (record.getLowerLimit() >= record.getUpperLimit()) {
            throw new IntegralException(
                String.format("Нижний предел (%.6f) должен быть меньше верхнего
(%.6f)", record.getLowerLimit(), record.getUpperLimit()));
        }

        // Проверка шага
        if (record.getStep() <= 0) {
            throw new IntegralException("Шаг должен быть положительным
числом");
        }

        if (record.getStep() > record.getUpperLimit() - record.getLowerLimit())
{
            throw new IntegralException(
                String.format("Шаг (%.6f) больше разницы пределов (%.6f)",
record.getStep(), record.getUpperLimit() - record.getLowerLimit()));
        }
    }

    private static void validateValues(double step, double upperLimit, double
lowerLimit)
        throws IntegralException {
        if (step < RecIntegral.MIN_VALUE || step > RecIntegral.MAX_VALUE) {
            throw new IntegralException("Шаг вне допустимого диапазона",
"step", step);
        }
        if (upperLimit < RecIntegral.MIN_VALUE || upperLimit >
RecIntegral.MAX_VALUE) {
            throw new IntegralException("Верхний предел вне допустимого
диапазона", "upperLimit", upperLimit);
        }
        if (lowerLimit < RecIntegral.MIN_VALUE || lowerLimit >
RecIntegral.MAX_VALUE) {
            throw new IntegralException("Нижний предел вне допустимого
диапазона", "lowerLimit", lowerLimit);
        }
    }
}

```

